

현재 구현 · 반드시 구현 · 구현되면 좋은 기능을 분리하고, 왜 이 API 경계가 필요한지 UX·ERD·AI·운영 관점에서 설명

설계 한 줄 요약 · FITLY의 API는 화면 버튼을 그대로 endpoint로 복제하는 것이 아니라, **사용자 행동의 경계 + 데이터 책임 + 트랜잭션 경계**를 기준으로 나눈다. Customer는 내 옷 → AI 추천 → 대여, Enterprise는 사진 → AI 초안 → 검토 → 상품/재고, Admin은 주문 → 반납 → 검수 → 세탁 → 재고복구 흐름을 가진다. AI는 추천/초안을 만들지만 상품 Variant·가격·재고·정산의 최종 사실은 PostgreSQL과 Spring Service가 결정한다.

Vue / Actor UX Customer · Admin · Enterprise	REST API Request/Response 계약	Spring Service 권한 · BR · Transaction	PostgreSQL Variant · Stock · Snapshot	Spring AI Closed Set Reasoning	Structured Output 검증 후 UI/DB 연결
---	---	---	--	---	--

1. 문서 기준과 API 명세 작성 원칙

참조 기준

- **현재 구현 상태**: 첨부 backend.zip Spring Boot 소스 스냅샷
- **서비스 기능**: FITLY Wireframe/UX 설계 전략
- **데이터 계약**: FITLY ERD 및 운영 전략
- **AI 경계**: FITLY AI 아키텍처 및 운영 전략
- **문서 형식**: Cobinding API 명세 가이드 - Overview, Getting Started, Request Syntax/Header/Element, Response, Common Error 구조
- **Repository 식별자**: <https://github.com/w-000000/fitly>

검증 범위: 생성 시점에 GitHub URL 자체를 직접 가져오지 못해, endpoint의 “현재 구현 여부”는 첨부 backend.zip을 Source of Truth로 판정했다.

상태 표기

표기	의미
✅ 구현	현재 코드에 endpoint와 동작이 존재하며 최종 설계와 대체로 정합
🟡 구현+보완	endpoint는 있으나 ERD/보안/운영/최종 UX에 맞게 수정 필요
🔴 필수	실서비스 또는 최종 Wireframe 완성에 필요하지만 현재 없음
🔵 권장	MVP 후 운영 효율/확장성에 유용
⚪ Legacy	현재 코드에는 있으나 최종 계약과 중복되어 제거/비노출 권장

핵심 원칙: “구현되어 있다”와 “최종 API로 유지해야 한다”는 다르다. 이 문서는 두 상태를 의도적으로 분리한다.

2. 현재 Backend에서 확인된 기술/운영 상태

Spring Boot 3.4.2 Java 21 · Maven · Spring Data JPA · Validation	Spring AI 1.1.8 OpenAI starter · ChatClient · typed entity output	후보 최대 5개 size · stock · status · budget DB hard filtering
AI Provider FITLY_AI_PROVIDER=mock openai. 기본값 mock으로 CI/로컬 API 비용 차단.	Retry / Failure AI output/일시 오류는 최대 2회 시도 후 502/503 계열로 변환.	API 문서 Swagger UI와 OpenAPI JSON이 이미 노출됨.
동봉된 Surefire 리포트에는 총 20개 테스트, 실패 0 / 오류 0 결과가 남아 있다. 다만 현재 실행 환경에서는 Maven wrapper 의존성 다운로드가 차단되어 재실행 검증은 수행하지 못했다.		

3. 공통 API 계약 - 현재와 목표를 분리

현재 구현 계약

항목	현재
Base URL	/api
인증	X-User-Id, X-Actor-Role 신뢰 헤더
Role	ROLE_CUSTOMER, ROLE_PARTNER, ROLE_ADMIN
성공 응답	DTO/Entity raw JSON
오류 응답	{timestamp,status,message}
AI 오류	Invalid response → 502, upstream/config → 503
Multipart	Wardrobe image 최대 10MB

목표 v1 계약

항목	목표
인증	Authorization: Bearer <token>; userId/role은 서버에서 해석
Role	ERD와 동일한 CUSTOMER / ENTERPRISE / ADMIN 의미로 통일
성공/실패	HTTP Status가 1차 의미, Body는 공통 code/message/result
금전/재고	POST Rental에 Idempotency-Key 권장/필수
시간	ISO-8601, timezone 명시
페이지네이션	목록 API는 page,size,sort 또는 cursor 표준화

```
{
  "code": "RECOMMENDATION_CREATED",
  "message": "추천을 생성했습니다.",
  "result": { ... }
}
```

왜 인증 교체가 필수인가? 현재 헤더는 프론트가 임의로 userId/role을 바꿀 수 있어 실제 서비스의 권한 경계가 아니다. ERD의 USER_ROLE/BUSINESS_MEMBER 제약을 API 보안과 연결하려면 서버가 인증 주체에서 사용자와 권한을 해석해야 한다.

4. Getting Started - Actor별 API 사용 시퀀스

Customer

- 1. 로그인/프로필
- 2. 내 옷 등록 또는 옷장 조회
- 3. AI 인식값 확인·수정
- 4. 추천 생성
- 5. 추천 상세/저장/피드백
- 6. 추천 기반 대여 생성
- 7. 대여 현황 조회
- 8. 반납 신청 또는 소장

UX 이유: Wireframe의 Primary CTA를 그대로 유지하되, 화면 state가 서버의 persisted state와 분리되지 않게 한다.

Enterprise

- 1. 로그인 → BusinessMember 확인
- 2. 상품 이미지 Storage 업로드
- 3. Local Vision 분석
- 4. AI Description 초안 생성
- 5. MD가 Textarea 수정
- 6. 상품 생성/수정
- 7. Variant·재고 관리
- 8. 정산 이력 조회

UX 이유: AI가 상품을 자동 확정하지 않고 Human-in-the-loop를 API 경계에서도 보장한다.

Admin

- 1. Dashboard 조회
- 2. 주문/반납 Queue 조회
- 3. 반납 확인
- 4. 검수 상태 업데이트
- 5. 세탁 완료
- 6. 재고 복구
- 7. 정산 생성/확정

운영 이유: RETURN_PROCESS의 상태 전이를 API로 그대로 드러내 장애 원인을 추적한다.

5. 현재 구현 API 전체 Inventory

아래 표는 backend.zip의 Controller annotation을 기준으로 작성했다. “”은 동작 여부와 별개로 최종 ERD/운영 설계에 맞추기 위한 보완이 필요함을 뜻한다.

Domain	Method	Path	권한	기능	상태	판정
Wardrobe	POST	/api/wardrobe/items	CUSTOMER	옷장 의류 등록 (multipart image + metadata)		구현됨. 현재 이미지를 DB data URI로 저장하므로 Storage URL + 업로드 시 Vision 1회 분석 구조로 리팩터 필요
Wardrobe	GET	/api/wardrobe/items	CUSTOMER	내 옷장 목록		유지
Wardrobe	GET	/api/wardrobe/items/{id}	CUSTOMER	옷장 항목 상세		유지
Wardrobe	GET	/api/wardrobe/items/{id}/image	CUSTOMER	옷장 이미지 조회		구현됨. Storage 도입 후 signed URL/redirect 또는 proxy 방식 결정 필요
Wardrobe	PATCH	/api/wardrobe/items/{id}	CUSTOMER	AI 인식값/메타데이터 보정		Human-in-the-loop 핵심 API로 유지
Wardrobe	DELETE	/api/wardrobe/items/{id}	CUSTOMER	옷장 항목 삭제		유지
Recommendation	POST	/api/recommendations	CUSTOMER	Spring AI 코디 추천 생성		최종 설계와 가장 잘 정렬된 핵심 API
Recommendation	POST	/api/recommendations/rules	Public	규칙 기반 추천		테스트/과도기용. 최종 프론트 계약에서는 비노출 또는 제거 권장
Recommendation	POST	/api/recommendations/jobs	CUSTOMER	Mock 추천 Job 생성		구형 Flow. 최종 Recommendation API와 중복
Recommendation	GET	/api/recommendations/jobs/{id}	CUSTOMER/ADMIN	구형 추천 Job 조회		최종 조회 API로 대체 권장
Recommendation	GET	/api/recommendations/jobs	CUSTOMER	구형 추천 Job 목록		최종 추천 이력 API로 대체 권장
Recommendation	GET	/api/recommendations/saved	CUSTOMER	저장 코디 목록		구현은 되어 있으나 OUTFIT_FEEDBACK 테이블을 저장 기능에 재사용하여 ERD 의미와 충돌
Recommendation	PUT	/api/recommendations/jobs/{jobId}/looks/{lookKey}/saved	CUSTOMER	코디 저장		ERD와 저장 의미 재설계 필요
Recommendation	DELETE	/api/recommendations/jobs/{jobId}/looks/{lookKey}/saved	CUSTOMER	코디 저장 해제		ERD와 저장 의미 재설계 필요
Product	GET	/api/products	Public	상품 목록		구현됨. 검색/상태/기업/페이지네이션 필터 필요
Product	GET	/api/products/{productId}	Public	상품 상세 + variants		유지
Product	POST	/api/products	PARTNER/ADMIN	상품 등록		구현됨. partnerId/settlementRate를 클라이언트가 직접 넘기는 구조는 인증 기반 Context로 변경 필요
Product	PATCH	/api/products/{productId}	PARTNER/ADMIN	상품 수정		소유 Business 검증 강화 필요
Product AI	POST	/api/products/ai-description	PARTNER	Spring AI 상품 설명 초안 생성		핵심 구현 완료. businessMemberId/VisionFeatures를 클라이언트가 신뢰 주입하지 않도록 리팩터 필요

Variant	POST	/api/products/{productId}/variants	PARTNER/ADMIN	사이즈/재고 Variant 생성	✓	유지
Variant	PATCH	/api/products/variants/{variantId}/stock	PARTNER/ADMIN	재고 증감	✓	동시성/소유권 검증 강화
Variant	GET	/api/products/variants/{variantId}/availability	Public	기간/수량 기준 가용성 조회	✓	유지
Rental	POST	/api/rentals	CUSTOMER	개인 대여 생성	●	구현됨. 현재 단일 variant 중심이며 sourceRecommendationId가 저장되지 않음
Rental	GET	/api/rentals/mine	CUSTOMER	내 대여 목록	✓	유지
Rental	GET	/api/rentals	ADMIN	전체 대여 목록 + 상태 필터	✓	검색/페이지네이션 추가 권장
Rental	GET	/api/rentals/{id}	CUSTOMER/ADMIN/ PARTNER	대여 상세	✓	Business 소유권 범위 검증 강화
Rental	GET	/api/rentals/partner/{partnerId}/revenue	PARTNER/ADMIN	기업 대여 매출 합계	●	현재 주문 집계값. 정식 정산과 분리해야 함
Rental	GET	/api/rentals/partner/{partnerId}/settlements	PARTNER/ADMIN	기업 정산 추정 합계	●	BUSINESS_SETTLEMENT 영속 데이터 조회로 대체 필요
Rental	POST	/api/rentals/{id}/return-request	CUSTOMER	반납 신청	✓	유지
Rental	POST	/api/rentals/{id}/rent-to-own	CUSTOMER	소장 전환	✓	유지
Group	POST	/api/group-rentals	CUSTOMER	단체 대여 신청	●	구현됨. requestedItems 자유문자열을 GROUP_RENTAL_REQUEST_ITEM 구조로 변경 필요
Group	GET	/api/group-rentals/mine	CUSTOMER	내 단체 대여 신청	✓	유지
Group	GET	/api/group-rentals	ADMIN	단체 대여 신청 전체	✓	상태/검색 필터 권장
Return/Laundry	POST	/api/laundry/inspections	ADMIN	검수 + 세탁완료 + 재고복구	●	구현됨. ERD의 return → inspect → laundry → restore 상태 흐름을 한 번에 압축함
Return/Laundry	GET	/api/laundry/inspections	ADMIN	검수/세탁 기록	✓	유지하되 상태 필터 필요
Dashboard	GET	/api/admin/dashboard	ADMIN	관리자 KPI/최근 주문	✓	Wireframe과 일치
Dashboard	GET	/api/partner/dashboard	PARTNER/ADMIN	기업 Dashboard	●	ERD의 ENTERPRISE 명칭과 ROLE_PARTNER 불일치 정리 필요
Ops	GET	/actuator/health	Ops	Health Check	✓	Spring Actuator로 노출
Docs	GET	/swagger-ui.html	Dev	Swagger UI	✓	springdoc-openapi 적용
Docs	GET	/v3/api-docs	Dev	OpenAPI JSON	✓	springdoc-openapi 적용

6. 핵심 API 상세 - Customer / Wardrobe

왜 필요한가? FITLY 추천의 입력은 단순 text가 아니라 “내가 실제로 가진 옷”이다. 따라서 이미지 등록 API는 파일 보관, Vision 분석, 사용자 보정까지 하나의 데이터 생명주기로 연결되어야 한다.

 구현+보완 **POST** **/api/wardrobe/items**

Request Syntax	multipart/form-data · parts: metadata, image
현재 Header	X-Actor-Role: ROLE_CUSTOMER, X-User-Id
목표 Header	Authorization: Bearer
제약	image 필수, image/*, 최대 10MB

```
metadata = {
  "name": "Black Slacks",
  "category": "BOTTOM",
  "color": "BLACK",
  "season": "ALL",
  "description": "내 검정 슬랙스"
}
```

현재 **Response**: id/name/category/color/season/description/imageUrl/createdAt/updatedAt.

필수 리팩터: 현재는 이미지 bytes를 Base64 data URI로 만들어 PostgreSQL image_url 컬럼에 저장한다. 설계 브리핑의 “무거운 이미지는 Storage, DB에는 URL” 원칙과 충돌한다. 목표는 Storage 업로드 → Local Vision 1회 → aiDetectedCategory/Color 저장 → 프론트가 PATCH로 보정하는 구조다.

 구현 **GET** **/api/wardrobe/items**

내 옷장 목록. 추천 입력에서 “내 옷장 불러오기”에 직접 사용한다.

 구현 **PATCH** **/api/wardrobe/items/{id}**

사용자 보정 API. Vision 오류가 LLM 추천 전체로 전파되는 것을 막는 Human-in-the-loop의 핵심이다.

7. 핵심 API 상세 - Customer AI Recommendation

왜 필요한가? 추천 API는 LLM에게 “면접 옷 추천해줘”를 전달하는 endpoint가 아니라, DB가 검증한 Variant 후보와 사용자 옷을 하나의 제한된 Context로 묶고 AI 결과를 다시 서버가 검증·저장하는 orchestration 경계다.

✓구현 **POST** **/api/recommendations**

Header	현재: X-Actor-Role, X-User-Id / 목표: Authorization Bearer
Content-Type	application/json
성공	201 Created
DB 선필터	Product ACTIVE + Variant ACTIVE + availableStock>0 + size 일치 + budget 이하 + 최대 3~5개

```
{
  "tpo": "INTERVIEW",
  "style": "Formal",
  "size": "M",
  "budget": 30000,
  "wardrobeItemIds": [101]
}
```

```
{
  "requestId": 1001,
  "status": "COMPLETED",
  "recommendations": [{
    "recommendationId": 2001,
    "rank": 1,
    "outfitTitle": "면접에 적합한 네이비 블레이저 코디",
    "matchScore": 85,
    "stylingComment": "...",
    "wardrobeItems": [{"wardrobeItemId":101,"itemName":"Black Slacks","cost":0}],
    "rentalItems": [{"productId":501,"productVariantId":1001,"productName":"Navy Blazer","brandName":"...", "size":"M", "rentalPrice":25000}],
    "totalRentalPrice": 25000
  }],
  "missingFields": []
}
```

현재 구현의 강점: AI가 반환한 wardrobe/variant ID가 실제 입력 후보 집합 안에 있는지 재검증하고, rank 중복/누락·ID 중복·budget 초과를 서버에서 차단한다. matchScore 85는 DB matching_score=0.8500으로 정규화되며, 가격은 AI가 아니라 DB rentalPrice로 다시 계산한다.

추가 필수 조회 계약

상태	Method / Path	이유
●	GET /api/recommendations/requests/{requestId}	POST 후 새로고침/재접속 시 DB 저장 결과로 Screen 03/04를 복구
●	GET /api/recommendations/requests?page=&size=	구형 jobs API를 최종 데이터 모델 기반 추천 이력으로 대체
●	PUT /api/recommendations/{recommendationId}/feedback	ERD OUTFIT_FEEDBACK과 정확히 매핑

8. 저장(♡) 기능의 API/ERD 충돌 - 구현 전에 결정 필요

현재 코드

SavedOutfit Entity가 실제로 outfit_feedback 테이블에 매핑되어 sentiment=LIKE, reason_code=lookKey, feedback_text=title+description을 저장한다.

따라서 /recommendations/saved와 save/unsave API는 “저장” 기능은 동작하지만 **Feedback** 테이블의 의미를 다른 목적으로 재사용한다.

최종 ERD / Wireframe

ERD의 OUTFIT_FEEDBACK은 Recommendation 1:0..1로 LIKE/DISLIKE와 feedback text를 저장한다. Wireframe의 ♡ 저장은 별도의 “보관” 의미다.

필수 의사결정: ① Wireframe의 저장 기능을 MVP에서 제외하거나, ② 전용 저장 모델을 추가하거나, ③ Feedback과 저장을 하나의 의미로 재정의해야 한다. 현재처럼 이름만 다르게 같은 테이블을 공유하는 것은 추천하지 않는다.

9. 핵심 API 상세 - Enterprise Product + AI Description

왜 필요한가? Enterprise의 핵심 작업은 “AI가 상품을 등록”하는 것이 아니라 “사진/기본정보에서 Draft를 만들고 MD가 검토한 뒤 확정”하는 것이다. API도 생성 Draft와 최종 PRODUCT.description을 분리해야 한다.

● 구현+보완 POST /api/products/ai-description

```
{
  "businessId": 10,
  "businessMemberId": 21,
  "productId": null,
  "inputImageUrl": "https://storage/...",
  "product": {
    "productName": "Navy Single Blazer",
    "brandName": "MUSINSA STANDARD",
    "category": "OUTER",
    "providedInformation": "소재 정보가 있다면 기업이 직접 제공"
  },
  "visionFeatures": {
    "category": "OUTER", "primaryColor": "NAVY", "fit": "REGULAR",
    "silhouette": "STRAIGHT", "pattern": "SOLID", "confidence": 0.94
  }
}
```

```
{
  "generationId": 301,
  "status": "COMPLETED",
  "generatedDescription": "[헤드카피]..."
}
```

필수 보완: 현재 Request가 businessId/businessMemberId/VisionFeatures를 클라이언트에서 받는다. 실서비스에서는 로그인 사용자 → BusinessMember를 서버가 조회하고, Vision 결과도 서버가 생성/저장한 값 또는 신뢰 가능한 analysisId를 참조해야 한다.

● 구현+보완 POST /api/products

MD가 AI Draft를 수정한 뒤 최종 Product를 등록한다. 현재 partnerId와 settlementRate까지 Request에서 받으므로, Business/Contract는 인증 Context와 DB의 ACTIVE contract에서 해석하도록 바꾸는 것이 안전하다.

✓ 구현 POST /api/products/{productId}/variants

실제 대여 가능 단위는 Product가 아니라 size별 PRODUCT_VARIANT이므로 별도 endpoint가 타당하다.

✓ 구현 PATCH /api/products/variants/{variantId}/stock

Enterprise Inventory 화면과 연결. 단, Business 소유권 검증과 동시성 정책은 강화해야 한다.

10. 핵심 API 상세 - Rental / Return / Group / Settlement

개인 대여

 **POST** **/api/rentals**

현재는 variantId + quantity 하나를 받아 즉시 재고 reserve 후 RentalOrder를 생성한다.

```
{"variantId":1001,"quantity":1,"startDate":"2026-09-12","endDate":"2026-09-16","shippingAddress":"..."}
```

필수 변경 이유: 최종 추천 한 개는 여러 RECOMMENDATION_ITEM을 가질 수 있고 RENTAL_ORDER도 1:N RENTAL_ITEM이다. 따라서 sourceRecommendationId와 items[]를 저장하거나, recommendationId만 받고 서버가 rental items를 확정해야 추천 → 대여 추적성이 보존된다.

반납/소장

- ✓ POST /api/rentals/{id}/return-request
- ✓ POST /api/rentals/{id}/rent-to-own

Admin Return Process

현재 POST /api/laundry/inspections는 order 단위로 검사하고 cleaned=true이면 재고까지 즉시 복구한다.

Wireframe과 ERD는 반납 → 검수 → 세탁 → 재고복구를 서로 다른 상태로 추적한다. 운영 화면에서 “어디에서 멈췄는지” 알아야 하므로 실제 서비스에서는 단계별 상태 변경 API가 더 적합하다.

- GET /api/admin/returns?
returnStatus=&inspectionStatus=&laundryStatus=
- PATCH /api/admin/returns/{returnProcessId}

Group Rental

현재 requestedItems가 자유문자열이다. ERD의 GROUP_RENTAL_REQUEST_ITEM을 쓰려면 items=[category, quantity]로 구조화해야 한다.

Settlement

현재 partner settlement endpoint는 주문의 settlementAmount 합산값이다. 실제 BUSINESS_SETTLEMENT 이력/상태(PENDING/SETTLED/FAILED) API가 필요하다.

11. 반드시 구현/정리해야 하는 API 및 Contract

Domain	Method	Target	기능	필요 이유
Auth	POST	/api/auth/signup	회원 가입	실서비스에서 X-User-Id를 신뢰할 수 없기 때문. USER/ROLE을 실제 인증 흐름과 연결
Auth	POST	/api/auth/login	로그인 및 토큰 발급	현재 X-Actor-Role/X-User-Id 헤더는 테스트 편의를 위한 신뢰 헤더이므로 위조 가능
Profile	GET	/api/users/me	내 프로필/기본 사이즈 조회	Wireframe의 평소 착용 사이즈와 사용자 기본값을 서버의 사용자 상태로 연결
Profile	PATCH	/api/users/me	기본 사이즈/프로필 수정	추천 입력 반복을 줄이고 USER_PROFILE과 연결
Wardrobe	POST	/api/wardrobe/items (refactor)	Storage 업로드 + Local Vision 1회 분석 + AI 인식값 저장	현재 Base64 이미지를 PostgreSQL에 저장하고 추천 시 Vision을 호출함. 비용/DB 용량/설계 원칙과 충돌
Recommendation	GET	/api/recommendations/requests/{requestId}	최종 추천 요청/결과 재조회	POST 응답을 잃어도 persisted RECOMMENDATION/ITEM을 기준으로 화면을 복구해야 함
Recommendation	GET	/api/recommendations/requests	추천 이력 목록	구형 /jobs가 아니라 최종 RECOMMENDATION_REQUEST 모델로 이력을 제공
Feedback	PUT	/api/recommendations/{recommendationId}/feedback	LIKE/DISLIKE 및 이유 저장	ERD의 OUTFIT_FEEDBACK 의미를 API에 정확히 연결. 현재 SavedOutfit이 feedback 테이블을 다른 의미로 사용
Saved Look	TBD	저장 기능 API/DB 계약 재결정	Wireframe의 ♥ 저장은 필요하지만 ERD에는 전용 SAVED_OUTFIT 테이블이 없음. Feedback과 의미를 섞으면 안 됨	
Product	GET	/api/products?businessId=&status=&q=&page=	상품 목록 검색/필터/페이지네이션	Admin/Enterprise Table은 데이터가 늘면 전체 findAll 응답으로 운영하기 어려움
Product Image	POST	/api/products/images	상품 이미지 업로드 및 Storage URL 발급	Enterprise Drag&Drop UX와 DB URL 저장 원칙을 연결
Product AI	POST	/api/products/ai-description (refactor)	인증 사용자→BusinessMember 해석 + 서버 Vision 결과 사용	클라이언트가 businessMemberId와 VisionFeatures를 위조하지 못하도록 Source of Truth를 서버로 이동
Rental	POST	/api/rentals (refactor)	sourceRecommendationId + rentalItems[] 기반 주문	추천은 1:N 아이템인데 현재 API는 단일 variant 중심. 추천→실제 주문 추적성 필요
Rental	POST	/api/rentals with Idempotency-Key	중복 결제/더블 클릭 방지	대여 생성은 금전/재고 변경 API이므로 재시도 시 중복 주문 방지가 필요
Return	GET	/api/admin/returns	반납/검수/세탁 작업 큐	Admin Wireframe의 Returns 화면을 실제 상태 기준으로 구성
Return	PATCH	/api/admin/returns/{returnProcessId}	return/inspection/laundry 상태 단계 변경	ERD의 상태 전이를 하나의 최종 inspect 호출이 아니라 운영 단계별로 추적

Settlement	GET	/api/enterprise/settlements	실제 BUSINESS_SETTLEMENT 조회	현재 /rentals/.../settlements는 계산값일 뿐 정산 이력 테이블과 연결되지 않음
Settlement	POST	/api/admin/settlements	정산 기간 생성/확정	계약 이력과 snapshot을 기준으로 정산을 영속화해야 Enterprise Settlement 화면 완성
Group	POST	/api/group-rentals (refactor)	items:[{category,quantity}] 구조화	ERD의 GROUP_RENTAL_REQUEST_ITEM과 일치시키고 운영자가 자유 문장을 파싱하지 않게 함
Role naming	Contract	ROLE_PARTNER → ENTERPRISE 정합화	ERD USER_ROLE과 Wireframe Actor는 ENTERPRISE인데 현재 코드 enum은 ROLE_PARTNER	
Common Response	Contract	code/message/result 공통 응답 + errorCode 표준화	프론트가 endpoint별 예외 형태를 따로 해석하지 않게 하고 Tistory 가이드의 공통 계약을 반영	

12. 구현되면 좋은 API - 지금 만들지 않아도 되는 이유까지

Domain	Method	Target	기능	도입 판단
Try-on AI	POST	/api/try-on/jobs	선택한 코디의 가상 착용 이미지 생성 Job	사용자가 눌렀을 때만 호출. 생성 이미지 DB 장기저장 금지, TTL URL 반환
Try-on AI	GET	/api/try-on/jobs/{jobId}	생성 상태/임시 URL 조회	이미지 생성 latency가 길어질 때 202 + polling으로 핵심 API timeout 분리
Recommendation	POST	/api/recommendations/{requestId}/regenerate	조건은 유지하고 추천 재생성	사용자가 결과를 마음에 들어 하지 않을 때 입력을 다시 작성하지 않게 함
AI Audit	GET	/api/admin/ai/metrics	AI latency/failure/token/cost 지표	운영 비용과 장애를 endpoint 단위로 추적
Enterprise AI	GET	/api/products/ai-description/history	상품 설명 생성 이력	재생성/수정 과정의 감사 추적과 Prompt 품질 비교
Inventory	GET	/api/enterprise/inventory/low-stock	재고 부족 목록	Enterprise Dashboard 운영 우선순위 지원
Admin Export	GET	/api/admin/rentals/export	주문/반납 CSV 내보내기	운영 규모가 커졌을 때 수작업 정리 비용 감소
API Versioning	Convention	/api/v1/...	프론트 계약이 안정된 뒤 breaking change를 격리	
Async/Event	Event	Kafka/Outbox	동시 주문/후처리가 실제 병목이 된 뒤 도입. MVP 선도입 금지	

확장 원칙: Kafka, 비동기 AI Job, 별도 AI metrics API 같은 기능은 “있으면 멋진 기술”이라서가 아니라 실제 latency/traffic/cost 병목이 확인될 때 추가한다. 이는 ERD·AI 브리핑의 “현재는 단순하게, 부하가 발생한 지점만 분리” 원칙과 동일하다.

13. 공통 Error Specification

Cobinding 가이드처럼 공통 에러를 한 섹션에서 관리하고 각 API가 이를 참조하도록 한다. 현재 구현의 메시지 기반 응답을 아래 code 기반 계약으로 고정하는 것을 권장한다.

HTTP	Code	상황	Frontend 처리
400	INVALID_REQUEST	필수값/날짜/size/TPO 오류	입력 필드 강조
401	UNAUTHORIZED	토큰 없음/만료	로그인 이동
403	FORBIDDEN	다른 사용자의 옷/다른 Business 접근	권한 안내
404	RESOURCE_NOT_FOUND	상품/Variant/추천/주문 없음	목록으로 복귀
409	STOCK_CONFLICT	동시 주문으로 재고 부족/상태 전이 충돌	재고 갱신 후 재선택
413	IMAGE_TOO_LARGE	10MB 초과	압축/재업로드
415	UNSUPPORTED_MEDIA_TYPE	이미지 아닌 파일	파일 타입 안내
502	AI_INVALID_RESPONSE	Structured Output 파싱/검증 실패	일반 실패 UI + 재시도
503	AI_UNAVAILABLE	Provider timeout/config/upstream	잠시 후 재시도

```
{
  "code": "STOCK_CONFLICT",
  "message": "대여 가능한 재고가 변경되었습니다.",
  "result": null
}
```


14. API 운영·비용·배포 관점의 설계 이유

운영

- AI endpoint는 전체 상품을 받지 않고 DB가 최대 3~5개 Candidate를 선별
- AI가 반환한 variant ID/가격을 서버가 재검증
- 금전/재고 변경 API는 transaction + lock + idempotency로 보호
- 목록 API는 전체 findAll에서 pagination/search로 전환
- Actor별 API 응답은 필요한 정보만 노출하고 JPA Entity raw 반환은 점진적으로 DTO화

배포

- Core는 Vue + Spring Boot + Spring AI + PostgreSQL 단순 구조 유지
- Swagger/OpenAPI를 코드와 명세의 검증 도구로 활용
- 실제 Local Vision이 무거워질 때 Provider 경계를 유지한 채 FastAPI/GPU 서비스 분리
- API versioning은 frontend 계약 안정 후 적용

비용

- Wardrobe/Product 이미지는 Storage로 분리해 PostgreSQL Base64 용량 증가 방지
- Local Vision은 업로드 시 1회, 추천마다 반복하지 않음
- Spring AI LLM은 짧은 JSON Candidate만 전송
- Mock Provider로 CI/로컬의 외부 AI 호출 비용 0원 유지
- Try-on은 사용자 클릭 시에만 호출하고 TTL 이미지로 폐기

왜 이 경계가 타당한가?

Wireframe은 화면 수를 최소화하고, ERD는 책임/변경주기를 분리하며, AI 설계는 DB와 AI의 책임을 분리한다. API는 이 세 설계 사이의 계약이므로 **UI 버튼 수가 아니라 도메인 상태 변화**를 기준으로 나누어야 한다.

결론: “호출 가능한 API가 많다”가 좋은 명세가 아니다. 실제 상태를 변경하는 경계를 정확히 정의하고, 조회/생성/상태전이/AI 호출의 책임을 혼합하지 않는 것이 FITLY의 운영 비용과 장애 비용을 줄인다.

15. 구현 우선순위 제안

Priority	작업	완료 조건
P0	Auth + role naming + Business ownership	신뢰 헤더 제거, ENTERPRISE/ADMIN/CUSTOMER 권한이 DB와 연결
P0	Wardrobe image Storage + Vision 1회 분석	DB Base64 제거, AI 인식값 반환/수정 가능
P0	Recommendation 조회/이력 + Save/Feedback 의미 결정	최종 ERD 기반 Screen 03/04 복구 가능
P0	Rental refactor	sourceRecommendationId + multiple RentalItem + idempotency
P1	Enterprise image/description trust boundary	BusinessMember/Vision을 서버가 검증
P1	Return Process 단계 API	Admin Table에서 상태별 Queue 운영 가능
P1	Settlement persistence API	BUSINESS_SETTLEMENT 화면/이력 연결
P1	Pagination/filter/common response	프론트 계약 고정 및 대량 데이터 대비
P2	Try-on/AI metrics/export/versioning	실제 사용량/운영 요구가 확인된 뒤 도입